

Memory, vault, CASS, Smart Memory MCP

Individual printable topic report.

Generated 2026-06-17

Why I built it

Long-running agents need more than a large context window. They need raw memory, compressed memory, procedural learning, contradiction handling, retrieval, and a way to avoid acting on stale facts.

Architecture and evidence

``cass_pipeline.py``, ``memory_broker.py``, vault-watcher design notes, and ``smart-memory-mcp/server.js`` show schemas, scoring, contradiction logic, TF-IDF retrieval, recency/usefulness metadata, and MCP tool declarations.

Claim boundary: Presented as practical memory infrastructure, not proof of human-like consciousness.

- CASS keeps episodic raw records, structured working memory, and procedural rules with confidence/decay.
- Memory Broker broadcasts high-value or central facts across memory systems and flags contradictions.
- Vault watcher design handles file-change detection, incremental reindexing, graph updates, staleness propagation, and forgetting scores.
- Smart Memory MCP provides a simple local protocol component for learn/recall/stats/patterns/suggestions.

What I can show with this

- Memory was designed as structure, not just as a folder of notes.
- The system distinguishes raw episodes, compressed working memory, and procedural rules.
- Contradiction and staleness were treated as first-class risks.
- A small MCP component makes the memory idea portable across agent clients.
- Agents can become dangerous or useless when they act on stale summaries.
- Raw memory should remain available when summaries become distorted.
- Contradictions should be preserved and surfaced instead of overwritten.
- Retrieval scoring needs more than text similarity; usefulness and recency matter.
- Add a public synthetic-memory demo.
- Benchmark retrieval and contradiction detection on non-private data.
- Expose memory provenance in every agent response.
- Add explicit redaction rules for private vault content.